

SQL & EXCEL for Data Analytics

Task 1 – Introduction to Databases & SELECT

The SELECT statement is used to retrieve data from the Employees table. It can display all columns or only selected columns, and aliases can be used to provide meaningful names to the columns.

SQL Query:

```
DROP DATABASE IF EXISTS company_db;
CREATE DATABASE company_db;
USE company_db;
CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    department VARCHAR(30),
    salary INT,
    city VARCHAR(30)
);
INSERT INTO Employees (emp_id, emp_name, department, salary, city)
VALUES
(1, 'Rahul', 'IT', 60000, 'Hyderabad'),
(2, 'Priya', 'HR', 45000, 'Vijayawada'),
(3, 'Arjun', 'IT', 75000, 'Hyderabad'),
(4, 'Sneha', 'Finance', 55000, 'Chennai'),
(5, 'Kiran', 'Sales', 40000, 'Vijayawada'),
(6, 'Anjali', 'IT', 65000, 'Bangalore'),
(7, 'Ravi', 'Sales', 50000, 'Chennai'),
(8, 'Divya', 'HR', 48000, 'Hyderabad'),
(9, 'Manoj', 'Finance', 70000, 'Bangalore'),
(10, 'Neha', 'IT', 80000, 'Chennai');
SELECT * FROM Employees;
SELECT emp_id, emp_name, department, salary
FROM Employees;
SELECT
    emp_id AS Employee_ID,
    emp_name AS Employee_Name,
    department AS Department,
    salary AS Salary
FROM Employees;
```

Output:

	Employee_ID	Employee_Name	Department	Salary
▶	1	Rahul	IT	60000
	2	Priya	HR	45000
	3	Arjun	IT	75000
	4	Sneha	Finance	55000
	5	Kiran	Sales	40000
	6	Anjali	IT	65000
	7	Ravi	Sales	50000
	8	Divya	HR	48000
	9	Manoj	Finance	70000
	10	Neha	IT	80000

Task 2 – WHERE, ORDER BY & Aggregate Functions

WHERE is used to filter records based on specific conditions, while ORDER BY sorts the results in ascending or descending order. Aggregate functions such as COUNT, SUM, AVG, MIN, and MAX are used to calculate useful summaries from the employee data.

SQL Query:

```
SELECT * FROM Employees WHERE department='IT';
```

```
SELECT * FROM Employees WHERE salary > 60000;
```

```
SELECT *
```

```
FROM Employees
```

```
WHERE city = 'Hyderabad' AND salary > 50000;
```

```
SELECT *
```

```
FROM Employees
```

```
ORDER BY salary ASC;
```

```
SELECT *
```

```
FROM Employees
```

```
ORDER BY salary DESC;
```

```
SELECT COUNT(*) AS Total_Employees
```

```
FROM Employees;
```

```
SELECT SUM(salary) AS Total_Salary
```

```
FROM Employees;
```

```
SELECT AVG(salary) AS Average_Salary
```

```
FROM Employees;
```

```
SELECT MIN(salary) AS Minimum_Salary
```

```
FROM Employees;
```

```
SELECT MAX(salary) AS Maximum_Salary
```

```
FROM Employees;
```

Output:

	emp_id	emp_name	department	salary	city
▶	1	Rahul	IT	60000	Hyderabad
	3	Arjun	IT	75000	Hyderabad
	6	Anjali	IT	65000	Bangalore
	10	Neha	IT	80000	Chennai
•	NULL	NULL	NULL	NULL	NULL

	emp_id	emp_name	department	salary	city
▶	3	Arjun	IT	75000	Hyderabad
	6	Anjali	IT	65000	Bangalore
	9	Manoj	Finance	70000	Bangalore
	10	Neha	IT	80000	Chennai
•	NULL	NULL	NULL	NULL	NULL

	emp_id	emp_name	department	salary	city
▶	1	Rahul	IT	60000	Hyderabad
	3	Arjun	IT	75000	Hyderabad

	emp_id	emp_name	department	salary	city
▶	5	Kiran	Sales	40000	Vijayawada
	2	Priya	HR	45000	Vijayawada
	8	Divya	HR	48000	Hyderabad
	7	Ravi	Sales	50000	Chennai
	4	Sneha	Finance	55000	Chennai
	1	Rahul	IT	60000	Hyderabad
	6	Anjali	IT	65000	Bangalore
	9	Manoj	Finance	70000	Bangalore
	3	Arjun	IT	75000	Hyderabad
	10	Neha	IT	80000	Chennai
•	NULL	NULL	NULL	NULL	NULL

	emp_id	emp_name	department	salary	city
▶	10	Neha	IT	80000	Chennai
	3	Arjun	IT	75000	Hyderabad
	9	Manoj	Finance	70000	Bangalore
	6	Anjali	IT	65000	Bangalore
	1	Rahul	IT	60000	Hyderabad
	4	Sneha	Finance	55000	Chennai
	7	Ravi	Sales	50000	Chennai
	8	Divya	HR	48000	Hyderabad
	2	Priya	HR	45000	Vijayawada
	5	Kiran	Sales	40000	Vijayawada
•	NULL	NULL	NULL	NULL	NULL

Total_Employees
▶ 10

Total_Salary
▶ 588000

Average_Salary
▶ 58800.0000

Minimum_Salary
▶ 40000

Maximum_Salary
▶ 80000

Task 3 – GROUP BY & HAVING

GROUP BY is used to organize employees into groups based on a common department. Aggregate functions can then be applied to each group, while HAVING filters the grouped results according to a specified condition.

SQL Query:

```
SELECT department, COUNT(*) AS Employee_Count
FROM Employees
GROUP BY department;
SELECT department, AVG(salary) AS Average_Salary
```

```

FROM Employees
GROUP BY department;
SELECT department, SUM(salary) AS Total_Salary
FROM Employees
GROUP BY department;
SELECT department, COUNT(*) AS Employee_Count
FROM Employees
GROUP BY department
HAVING COUNT(*) > 2;

```

Output:

department	Employee_Count	department	Average_Salary	department	Total_Salary
IT	4	IT	70000.0000	IT	280000
HR	2	HR	46500.0000	HR	93000
Finance	2	Finance	62500.0000	Finance	125000
Sales	2	Sales	45000.0000	Sales	90000

department	Employee_Count
IT	4

Task 4 – SQL JOINS

SQL JOINS are used to combine related information from the Employees and Departments tables. INNER JOIN returns matching records, LEFT JOIN keeps all employee records, and RIGHT JOIN keeps all department records.

SQL Query:

```

CREATE TABLE Departments (
    department_id INT PRIMARY KEY,
    department VARCHAR(30),
    manager VARCHAR(50)
);
INSERT INTO Departments (department_id, department, manager)
VALUES
(1, 'IT', 'Suresh'),
(2, 'HR', 'Lakshmi'),
(3, 'Finance', 'Ramesh'),
(4, 'Sales', 'Anil'),
(5, 'Marketing', 'Kavya');
SELECT
    e.emp_id,
    e.emp_name,
    e.department,
    e.salary,
    d.manager
FROM Employees e
INNER JOIN Departments d
ON e.department = d.department;
SELECT
    e.emp_id,
    e.emp_name,

```

```

e.department,
d.manager
FROM Employees e
LEFT JOIN Departments d
ON e.department = d.department;
SELECT
    e.emp_id,
    e.emp_name,
    e.department,
    d.manager
FROM Employees e
RIGHT JOIN Departments d
ON e.department = d.department;

```

Output:

	emp_id	emp_name	department	salary	manager
▶	1	Rahul	IT	60000	Suresh
	2	Priya	HR	45000	Lakshmi
	3	Arjun	IT	75000	Suresh
	4	Sneha	Finance	55000	Ramesh
	5	Kiran	Sales	40000	Anil
	6	Anjali	IT	65000	Suresh
	7	Ravi	Sales	50000	Anil
	8	Divya	HR	48000	Lakshmi
	9	Manoj	Finance	70000	Ramesh
	10	Neha	IT	80000	Suresh

	emp_id	emp_name	department	manager
▶	10	Neha	IT	Suresh
	6	Anjali	IT	Suresh
	3	Arjun	IT	Suresh
	1	Rahul	IT	Suresh
	8	Divya	HR	Lakshmi
	2	Priya	HR	Lakshmi
	9	Manoj	Finance	Ramesh
	4	Sneha	Finance	Ramesh
	7	Ravi	Sales	Anil
	5	Kiran	Sales	Anil
	NULL	NULL	NULL	Kavya

Task 5 – SQL Subqueries

Subqueries allow one SQL query to be used inside another query. They are useful for comparing employee salaries with values such as the average or maximum salary and for retrieving related information based on another query.

SQL Query:

```

SELECT emp_id, emp_name, department, salary
FROM Employees
WHERE salary > (
    SELECT AVG(salary)
    FROM Employees
);
SELECT emp_id, emp_name, department, salary

```

```

FROM Employees
WHERE salary = (
    SELECT MAX(salary)
    FROM Employees
);
SELECT emp_id, emp_name, salary
FROM Employees
WHERE department = (
    SELECT department
    FROM Employees
    WHERE emp_name = 'Rahul'
);

```

Output:

emp_id	emp_name	department	salary
1	Rahul	IT	60000
3	Arjun	IT	75000
6	Anjali	IT	65000
9	Manoj	Finance	70000
10	Neha	IT	80000
NULL	NULL	NULL	NULL

emp_id	emp_name	department	salary
10	Neha	IT	80000
NULL	NULL	NULL	NULL

emp_id	emp_name	salary
1	Rahul	60000
3	Arjun	75000
6	Anjali	65000
10	Neha	80000
NULL	NULL	NULL

Task 6 – Excel Data Formatting, Sorting & Filtering

Excel formatting improves the readability and presentation of employee data. Sorting organizes records based on selected columns, while filtering displays only the records that meet specific conditions.

Output:

	A	B	C	D
1	Employee_ID	Employee_Name	Department	Salary
2	1	Rahul	IT	60000
3	2	Priya	HR	45000
4	3	Arjun	IT	75000
5	4	Sneha	Finance	55000
6	5	Kiran	Sales	40000
7	6	Anjali	IT	65000
8	7	Ravi	Sales	50000
9	8	Divya	HR	48000
10	9	Manoj	Finance	70000
11	10	Neha	IT	80000

	A	B	C	D
1	Employee_ID ▾	Employee_Name ▾	Department ▾	Salary ▾
2	10	Neha	IT	80000
3	3	Arjun	IT	75000
4	9	Manoj	Finance	70000
5	6	Anjali	IT	65000
6	1	Rahul	IT	60000
7	4	Sneha	Finance	55000
8	7	Ravi	Sales	50000
9	8	Divya	HR	48000
10	2	Priya	HR	45000
11	5	Kiran	Sales	40000

	A	B	C	D
1	Employee_ID ▾	Employee_Name ▾	Department ▾	Salary ▾
2	10	Neha	IT	80000
3	3	Arjun	IT	75000
5	6	Anjali	IT	65000
6	1	Rahul	IT	60000

Task 7 – Conditional Formatting & Excel Functions

Conditional formatting highlights data that meets specific conditions, making important values easier to identify. IF, COUNTIF, and SUMIF functions are used to categorize data, count records based on conditions, and calculate totals for specific groups.

Output:

	A	B	C	D
1	Employee_ID ▾	Employee_Name ▾	Department ▾	Salary ▾
2	1	Rahul	IT	60000
3	2	Priya	HR	45000
4	3	Arjun	IT	75000
5	4	Sneha	Finance	55000
6	5	Kiran	Sales	40000
7	6	Anjali	IT	65000
8	7	Ravi	Sales	50000
9	8	Divya	HR	48000
10	9	Manoj	Finance	70000
11	10	Neha	IT	80000

	A	B	C	D	E
1	Employee_ID ▾	Employee_Name ▾	Department ▾	Salary ▾	Salary Category ▾
2	1	Rahul	IT	60000	Normal Salary
3	2	Priya	HR	45000	Normal Salary
4	3	Arjun	IT	75000	High Salary
5	4	Sneha	Finance	55000	Normal Salary
6	5	Kiran	Sales	40000	Normal Salary
7	6	Anjali	IT	65000	High Salary
8	7	Ravi	Sales	50000	Normal Salary
9	8	Divya	HR	48000	Normal Salary
10	9	Manoj	Finance	70000	High Salary
11	10	Neha	IT	80000	High Salary

Number Of IT Employees	Total IT Salary	
4	280000	

Task 8 – VLOOKUP & XLOOKUP

VLOOKUP and XLOOKUP are used to retrieve related information from the Departments table using the Department ID. These functions help connect employee records with their corresponding department details efficiently.

Output:

	A	B	C	
1	department_id ▾	department ▾	manager ▾	
2		1 IT	Suresh	
3		2 HR	Lakshmi	
4		3 Finance	Ramesh	
5		4 Sales	Anil	
6		5 Marketing	Kavya	
7				

	A	B	C	D	E	F	G	H
1	Employee_ID ▾	Employee_Name ▾	Department ID ▾	Department ▾	Salary ▾	Salary Category ▾	VLOOKUP Department ▾	XLOOKUP Department ▾
2	1	Rahul		1 IT	60000	Normal Salary	IT	IT
3	2	Priya		2 HR	45000	Normal Salary	HR	HR
4	3	Arjun		1 IT	75000	High Salary	IT	IT
5	4	Sneha		3 Finance	55000	Normal Salary	Finance	Finance
6	5	Kiran		4 Sales	40000	Normal Salary	Sales	Sales
7	6	Anjali		1 IT	65000	High Salary	IT	IT
8	7	Ravi		4 Sales	50000	Normal Salary	Sales	Sales
9	8	Divya		2 HR	48000	Normal Salary	HR	HR
10	9	Manoj		3 Finance	70000	High Salary	Finance	Finance
11	10	Neha		1 IT	80000	High Salary	IT	IT
12								

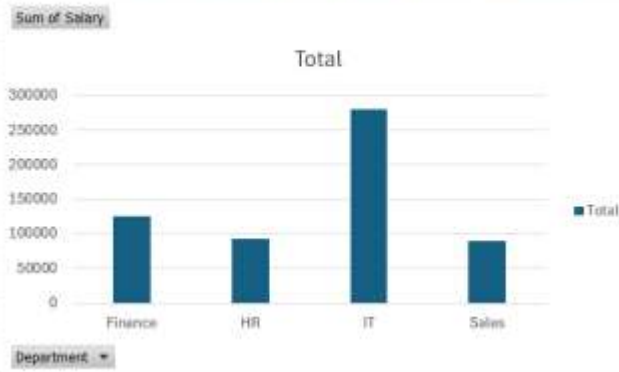
Task 9 – Pivot Table & Charts

The Pivot Table summarizes employee data by department and calculates values such as total salary. The Total Salary by Department chart provides a visual comparison of salary expenses across departments, making the summarized data easier to understand.

Output:

	A	B
1		
2		
3	Row Labels ▾	Sum of Salary
4	Finance	125000
5	HR	93000
6	IT	280000
7	Sales	90000
8	Grand Total	588000

Total Salary by Department



Explanation

The Total Salary by Department chart shows the total salary paid to employees in each department. It helps compare salary expenses across departments and makes it easy to identify which departments have higher or lower total salary costs.